



DATA ACQUISITION

I NEED DATA

1

Data Acquisition
Tools

2

Scraper

3

KAFKA

4

API

SOFTWARE TOOL

- Programming languages
 - Python, r, java,....
 - You have to write all code you need or use existing library
 - Pros and cons
- Data management platforms
 - KNIME
 - General purpose ML tool. It allows performing complex ML tasks via a drag and drop approach
 - TALEND
 - General purpose ETL (extract, transform, load) tool.
 - AIRFLOW
 - Python based orchestrator of data science pipeline
- Generative AI
 - The latest....
- There is no «one size fits for all» solution

KNIME: I/O TOOLS AND CSV COMPONENT

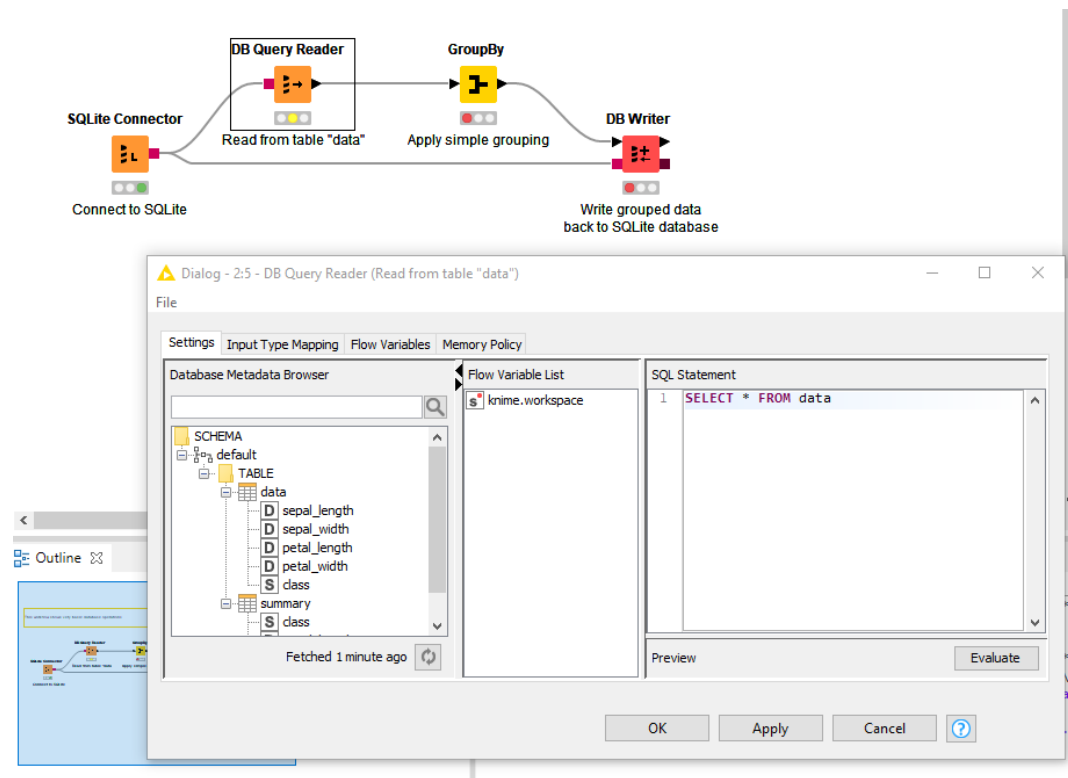
The screenshot displays the KNIME software interface, which is used for data science and machine learning workflows. The interface is divided into several main sections:

- Node Repository:** Located on the left, it shows a tree view of various data processing nodes. Under the 'IO' category, the 'Read' sub-category is expanded, listing nodes such as Excel Reader (XLS), File Reader, ARFF Reader, CSV Reader, Line Reader, Table Reader, PMML Reader, Model Reader, Fixed Width File Reader, List Files, Read Excel Sheet Names (XLS), Read Images, and Explorer Browser.
- Workflow Canvas:** The central area shows a workflow with a single node labeled 'CSV Reader' (Node 1) placed on a grid background.
- Dialog - 2:1 - CSV Reader:** A configuration window for the CSV Reader node is open on the right. It has tabs for 'Settings', 'Limit Rows', 'Encoding', 'Flow Variables', and 'Memory Policy'. The 'Settings' tab is active, showing the 'Input location' field with a 'Browse...' button and a 'Custom connection timeout [s]' set to 1. The 'Reader options' section includes fields for 'Column Delimiter' (comma), 'Row Delimiter' (newline), 'Quote Char' (double quote), and 'Comment Char' (hash). Checkboxes for 'Has Column Header' and 'Has Row Header' are both checked, and 'Support Short Lines' is unchecked.
- Editor:** At the bottom, a Python script is being edited in a file named 'result.py'. The script uses the 'csv' module to read a CSV file and print its contents.
- IPython console:** The console on the right shows the output of the script, displaying a list of data rows from the CSV file.

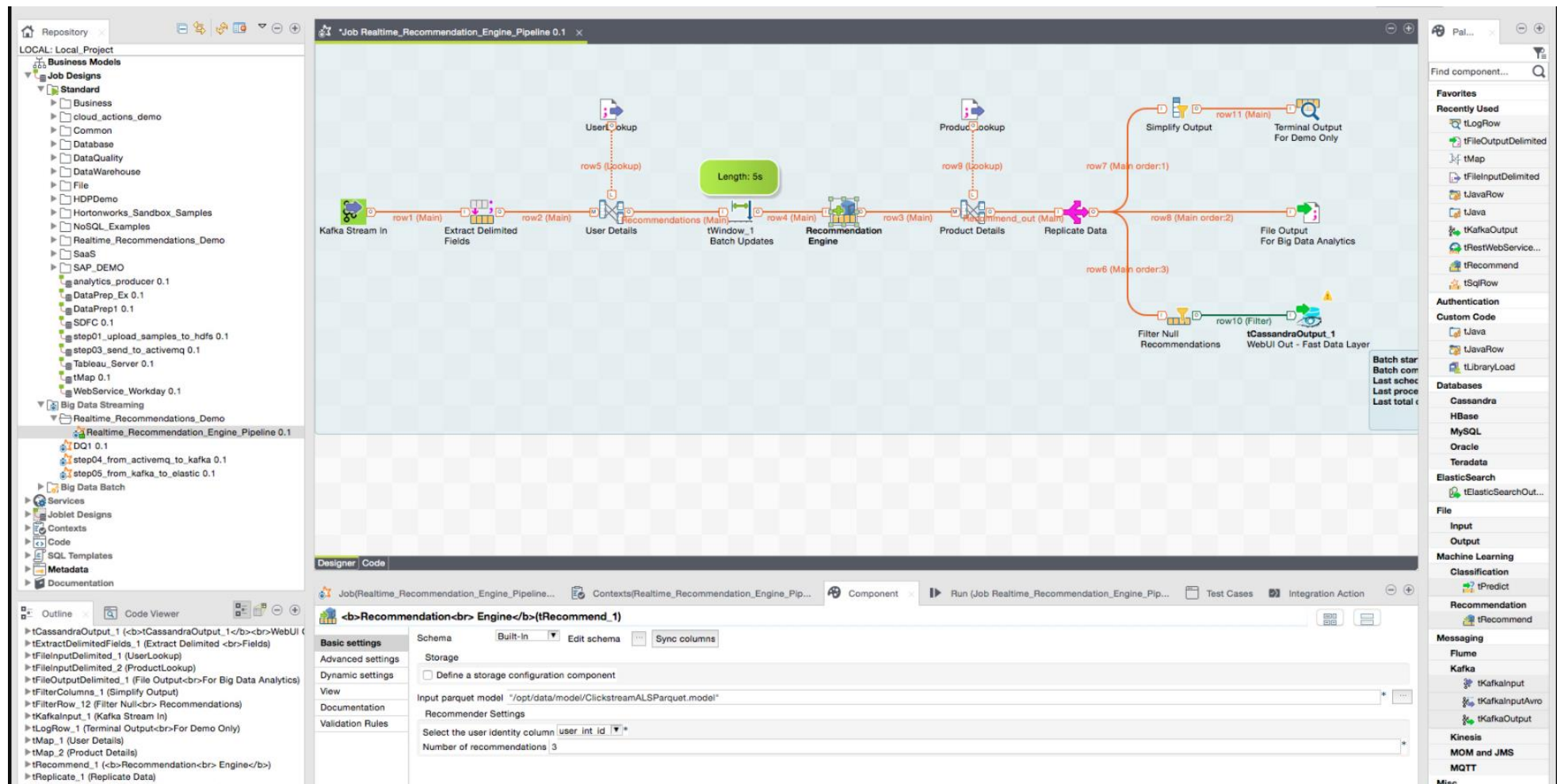
```
1 import csv
2 with open("/home/aqsa/Documents/sample1.csv", 'r') as file:
3     reader = csv.reader(file, delimiter = '\t')
4     for row in reader:
5         print(row)
6
7
8
```

```
In [11]: runfile('/home/aqsa/Desktop/result.py',
              wdir='/home/aqsa/Desktop')
['Serial_No,BOOK
Name,WRITER']
['1,Beauty and the
beast,Mathira']
['2,Cinderella,John Alix']
['3,Maat,Umera Ahmed']
['4,Harry potter,Maary
Jackson']
```

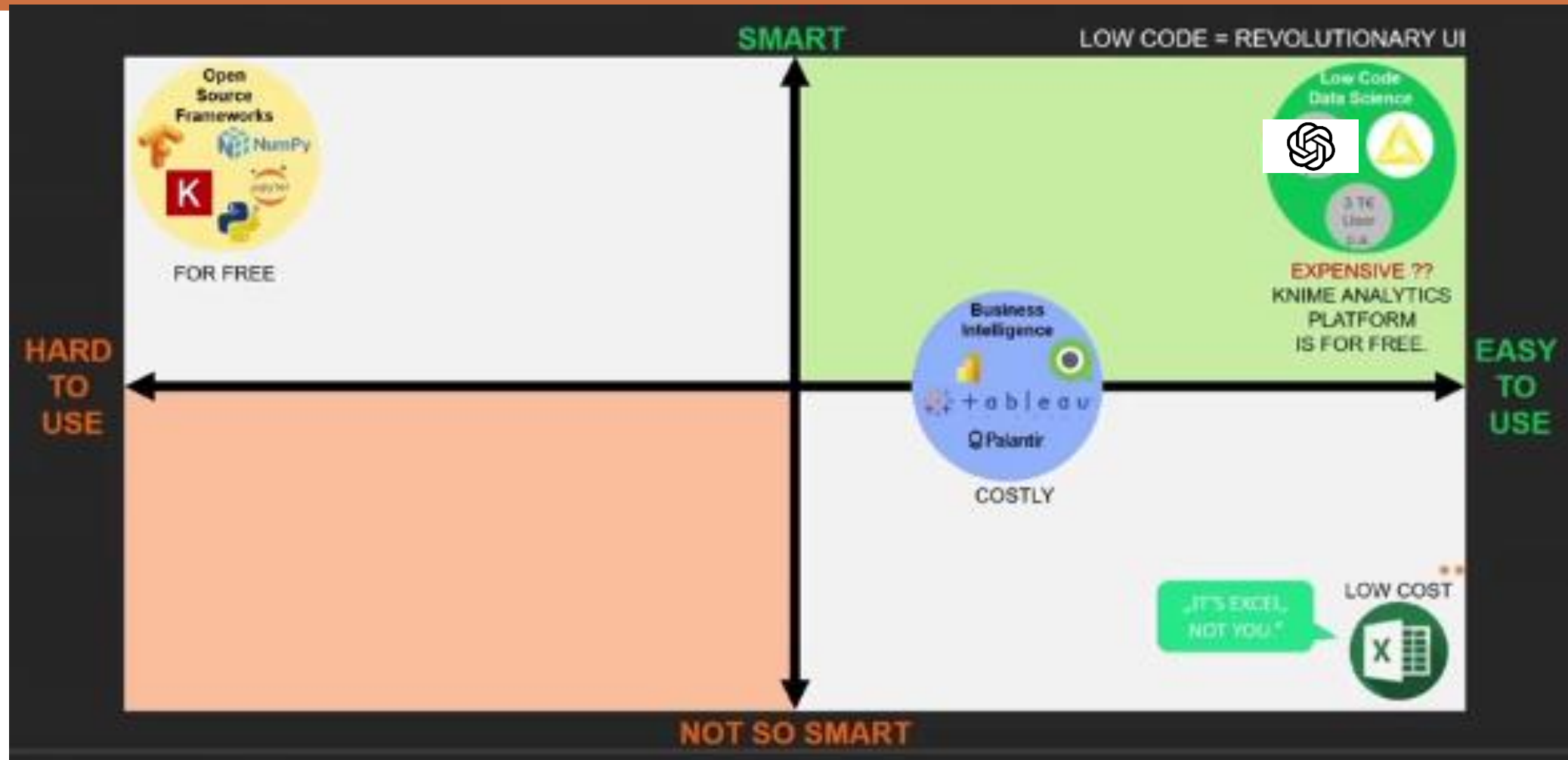
KNIME: SQL ACCESS



TALEND

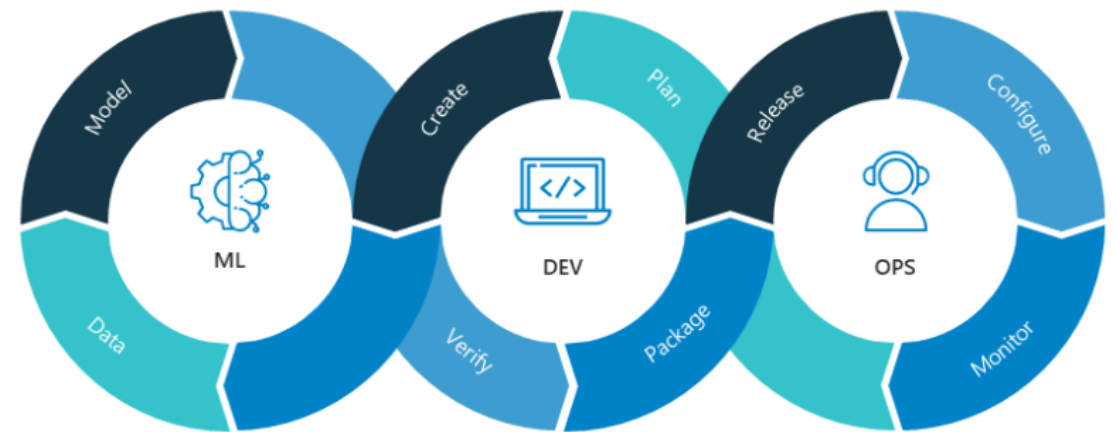


GENERAL TOOLS



WHEN USE TOOLS

- All pipeline steps are developed by means of a tool such as knime or talend
- When there are no special constraints in the data acquisition
- There is the need to apply the pipeline multiple times or «in production» environmental
- Such tools can be used also if you use API





DATA ACQUISITION TECHNIQUES





API REST



THE PROBLEM

- I need to get health information about European countries in 2020
- Where can I find data?
 - World health organization (www.who.int)
- I need to download data for further analysis

WHAT ARE API

- An API (Application Programming Interface) is a standardized and secure interface that allows applications to communicate and work with each other.
- API allows you to request data from a third-party data provider so that you can use this information as and when needed.
- API REST (REpresentational State Transfer) defines how applications can communicate over HTTP to transfer information efficiently and quickly.

REST CONSTRAINTS

Constraint	Property
Client-Server	Separation of concerns: simplicity, scalability, evolvability
Stateless	Scalability, reliability, visibility
Cacheable	Scalability, performance
Uniform Interface	Simplicity, evolvability, visibility
Layered System	Scalability, security, legacy integration
Identification of Resources	Simplicity, scalability. Required by uniform interface
Manipulation via representations	Evolvability. Required by uniform interface
Self-descriptive messages	Required by stateless, cacheable, uniform interface, and layered system
Hypermedia as the engine of application state	Scalability, reliability, evolvability, performance. Required by uniform interface

CRUD OPERATIONS

HTTP	Method	CRUD	Desc.
POST	CREATE	Create	-
GET	RETRIEVE	Retrieve	Safe,Idempotent,Cacheable
PUT	UPDATE	Update	Idempotent
DELETE	DELETE	Delete	Idempotent

API AUTHENTICATION

- Most APIs require authentication
 - API key, username, password, etc.,
- API Key includes a unique combination of characters and numbers that you need to send with every request to the server to retrieve data.
- Sometime are for free or for a limited used, in other situations you must pay
 - Pay as you go, other

ENDPOINT

- A REST API exposes a set of public URLs that client applications use to access the resources of a web service. These URLs, in the context of an API, are called endpoints.

HTTP method	API endpoint	Description
GET	/customers	Get a list of customers.
GET	/customers/<customer_id>	Get a single customer.
POST	/customers	Create a new customer.
PUT	/customers/<customer_id>	Update a customer.
PATCH	/customers/<customer_id>	Partially update a customer.
DELETE	/customers/<customer_id>	Delete a customer.

API USAGE

- How to use API
 - Find documentation
- For retrieving data
 - Use GET+parameters

GET `http://HOST[:PORT]/PATH/endpoint[?QUERY_PARAMETERS]`

- Output
 - Xml, JSON

JSON

- JSON (JavaScript Object Notation) is a lightweight data-interchange format
- JSON is based on two structures
 - A collection of name/value pairs (objects)
 - An ordered list of values (array)

```
{ "name": "John",  
  "age": 30,  
  "car": "ford",  
  "TelephoneNumber": ["555/3456", "555/654432"]  
}
```

```
{ "name": "John",  
  "age": 30,  
  "car": "ford",  
  "Telephone": [  
    {"Number": "555/3456", "role": "home"},  
    {"Number": "555/654432", "role": "office"}  
  ]  
}
```

API USAGE

- GET <https://dw.euro.who.int/api/v3/countries>

- Output

```
[  
  {"code":"ALB","iso2":"AL","iso3":"ALB","who_code":"ALB","short_name":"Albania","full_name":"Albania","sort_order":1},  
  {"code":"AND","iso2":"AD","iso3":"AND","who_code":"AND","short_name":"Andorra","full_name":"Andorra","sort_order":2},  
  {"code":"ARM","iso2":"AM","iso3":"ARM","who_code":"ARM","short_name":"Armenia","full_name":"Armenia","sort_order":3}  
  ,  
  ....  
]
```

API USAGE

- GET https://dw.euro.who.int/api/v3/measures/H2020_1?filter=COUNTRY:ITA
- {"code":"H2020_1","short_name":"Premature mortality","full_name":"Age-standardized overall premature mortality rate (from 30 to under 70 years) for four major non-communicable diseases (cardiovascular diseases, cancer, diabetes mellitus and chronic respiratory diseases)","updated_on":"2019-0814T07:31:22.963« ...
- {"COUNTRY":"ITA","COUNTRY_GRP":"","YEAR":"1997"},"value":{"display":"320.64","numeric":320.64}},{"fact_id":"31617080","attributes":{"MEASURE_TYPE":"AGE_STD_RATE"},"dimensions":{"COUNTRY":"ITA","COUNTRY_GRP":"","YEAR":"2000"},"value":{"display":"292.14","numeric":292.14}}

CHAT ON API

- Can you write python code for accessing GHO ODATA API about Italy in 2022
 - Try...ERROR
- The URL is out of date
- The correct URL is
 - <https://ghoapi.azureedge.net/api/>



To access data from the Global Health Observatory (GHO) using the OData API, you can use the `requests` library in Python. First, ensure that you have the necessary libraries installed by running:

bash

Copy code

```
pip install requests
```

Here's an example of how you can use Python to access GHO OData API for Italy in 2022:

python

Copy code

```
import requests

# Define the API endpoint for GHO OData
api_url = "https://apps.who.int/gho/athena/api/GHO/WHOSIS_000001?filter=COUN"

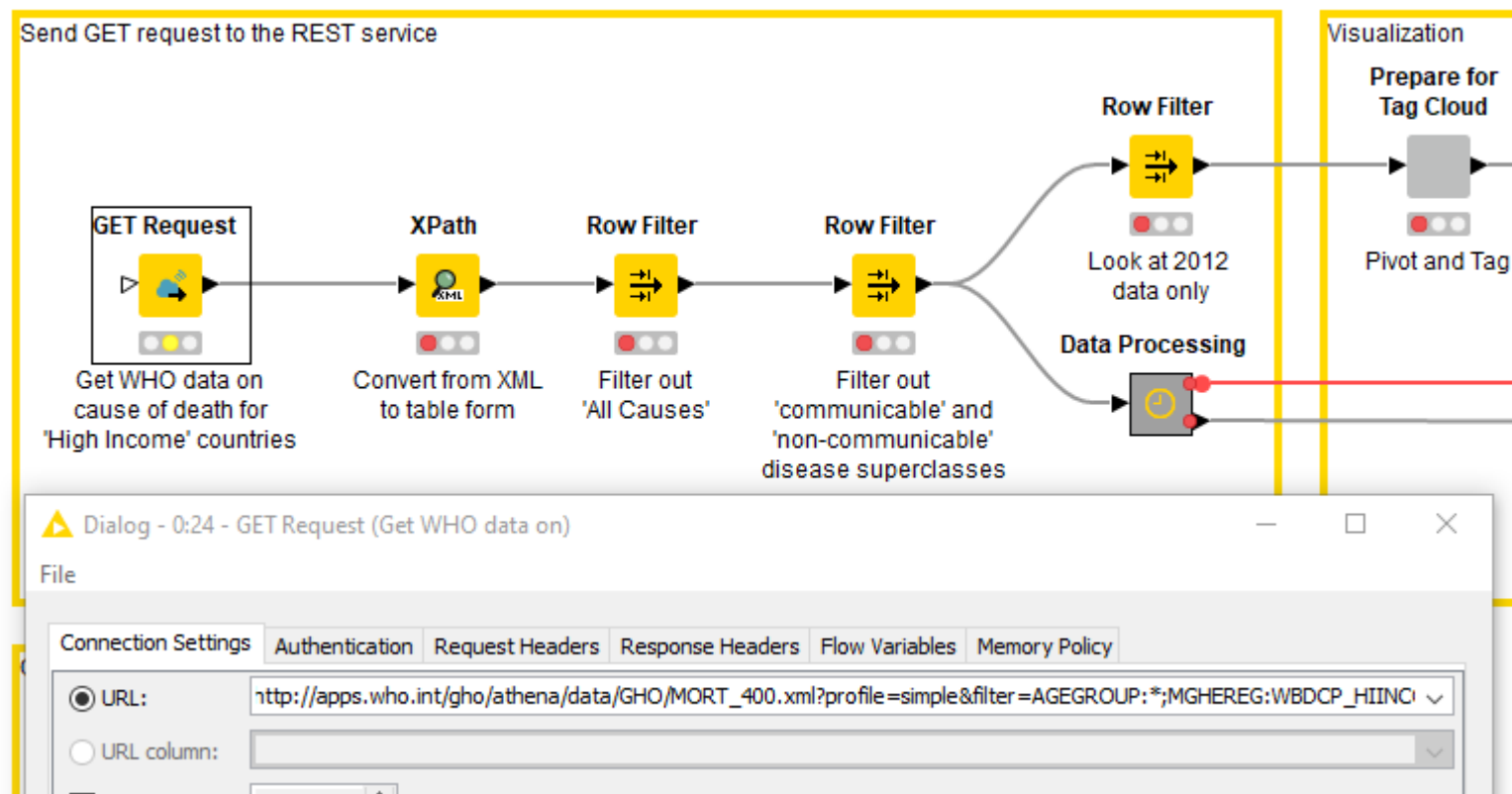
# Send a GET request to the API endpoint
response = requests.get(api_url)

# Check if the request was successful
if response.status_code == 200:
    # Parse the JSON response
    data = response.json()

    # Print the data (you can process it further as needed)
    print(data)
else:
    print("Failed to retrieve data. Status code:", response.status_code)
```

API USAGE

■ Tools: Knime



JSON QUERY

- To extract content from a JSON document there is the need for a query language
- There are no standard query languages for JSON
- KNIME adopts a «path expression» languages similar to XPATH
 - Remember that JSON is a tree
 - A query is expressed as all possible satisfied «paths» from root to leaves

JSON PATH

JSON document
`{"book": [`

`{"year": 1999,`

`"title": "Timeline",`

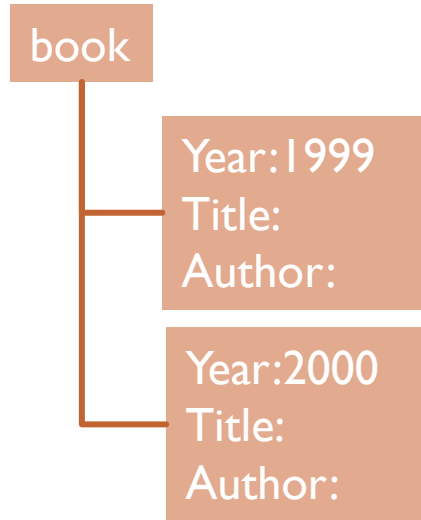
`"author": "Michael Crichton"},`

`{"year": 2000,`

`"title": "Plain Truth",`

`"author": "Jodi Picoult"}]`

`}`



Examples

- `$.book[0]`
 - `{"year": 1999, "title": "Timeline", "author": "Michael Crichton"}`
- `$.book[*].year`
 - `[1999, 2000]`
- `$.book[2].year`
 - `?`
- `$.book[?(@.year==1999)].title`
 - `Timeline`

API USAGE

- Python
- Library requests

```
In [1]: import requests
```

```
In [2]: api_url = "https://dw.euro.who.int/api/v3/countries"  
response = requests.get(api_url)  
response.json()
```

```
Out[2]: [{ 'code': 'ALB',  
            'iso2': 'AL',  
            'iso3': 'ALB',  
            'who_code': 'ALB',  
            'short_name': 'Albania',  
            'full_name': 'Albania',  
            'sort_order': 1},  
          { 'code': 'AND',  
            'iso2': 'AD',
```

```
In [7]: api_url="https://dw.euro.who.int/api/v3/measures/H2020_1"  
myParam={"filter":"COUNTRY:ITA"}  
response=requests.get(api_url,params=myParam)  
response.json()
```

```
Out[7]: { 'code': 'H2020_1',  
          'short_name': 'Premature mortality',  
          'full_name': 'Age-standardized overall premature mortality rate (from 30 to under 70 years) for four major non-communicable diseases (cardiovascular diseases, cancer, diabetes mellitus and chronic respiratory diseases)',  
          'updated_on': '2019-08-14T07:31:22.963',  
          'data_updated_on': '2018-06-15T08:06:14.6',  
          'metadata_updated_on': '2019-08-14T07:31:22.963',  
          'notes_updated_on': None,
```



«NEAR» REAL TIME DATA ACQUISITION



REAL-TIME/FAST DATA



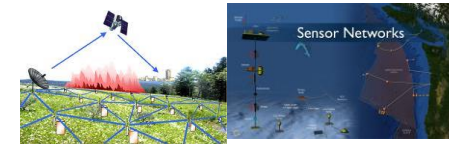
Social media and networks
(all of us are generating data)



Scientific instruments
(collecting all sorts of data)



Mobile devices
(tracking all objects all the time)



Sensor technology and networks
(measuring all kinds of data)

- The progress and innovation is no longer hindered by the ability to collect data
- But, by the ability to manage, analyze, summarize, visualize, and discover knowledge from the collected data in a timely manner and in a scalable fashion

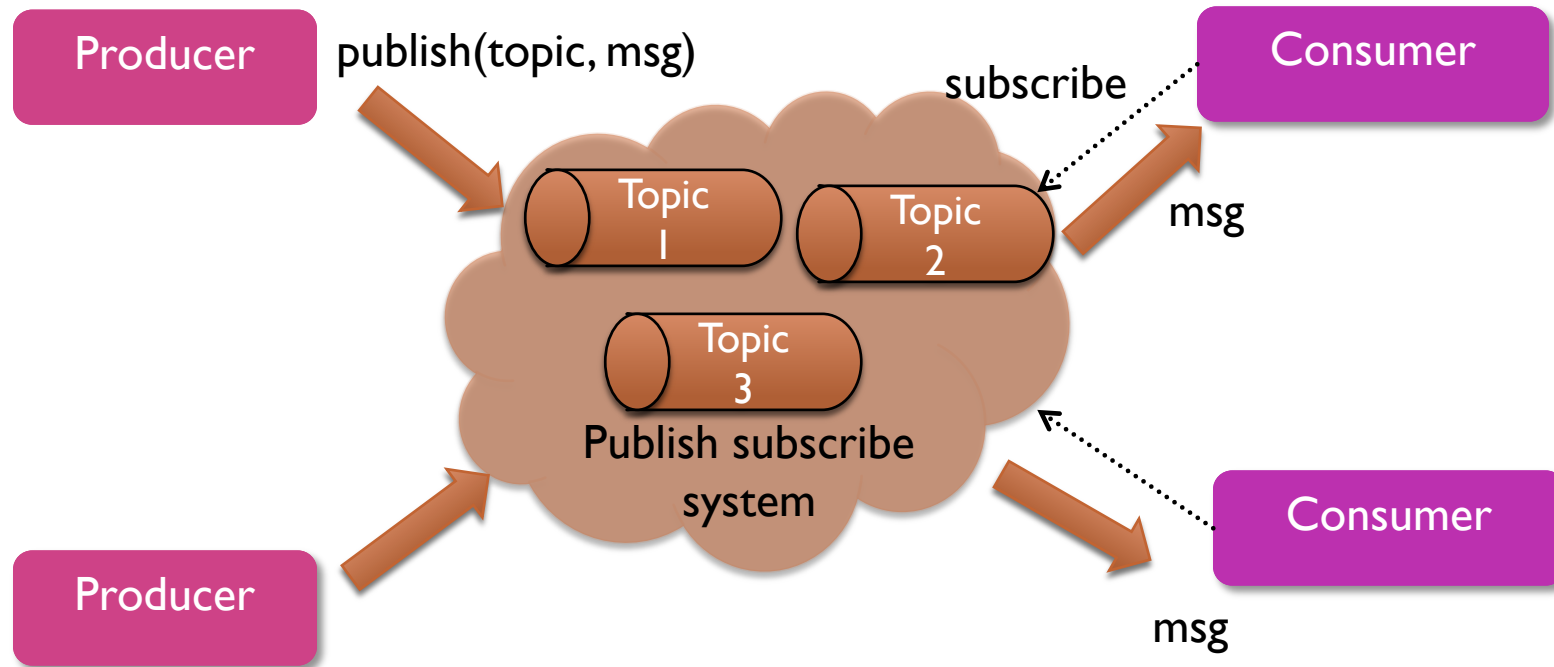
SOMETHING TO REMEMBER ...

Real-time $\neq$ FAST

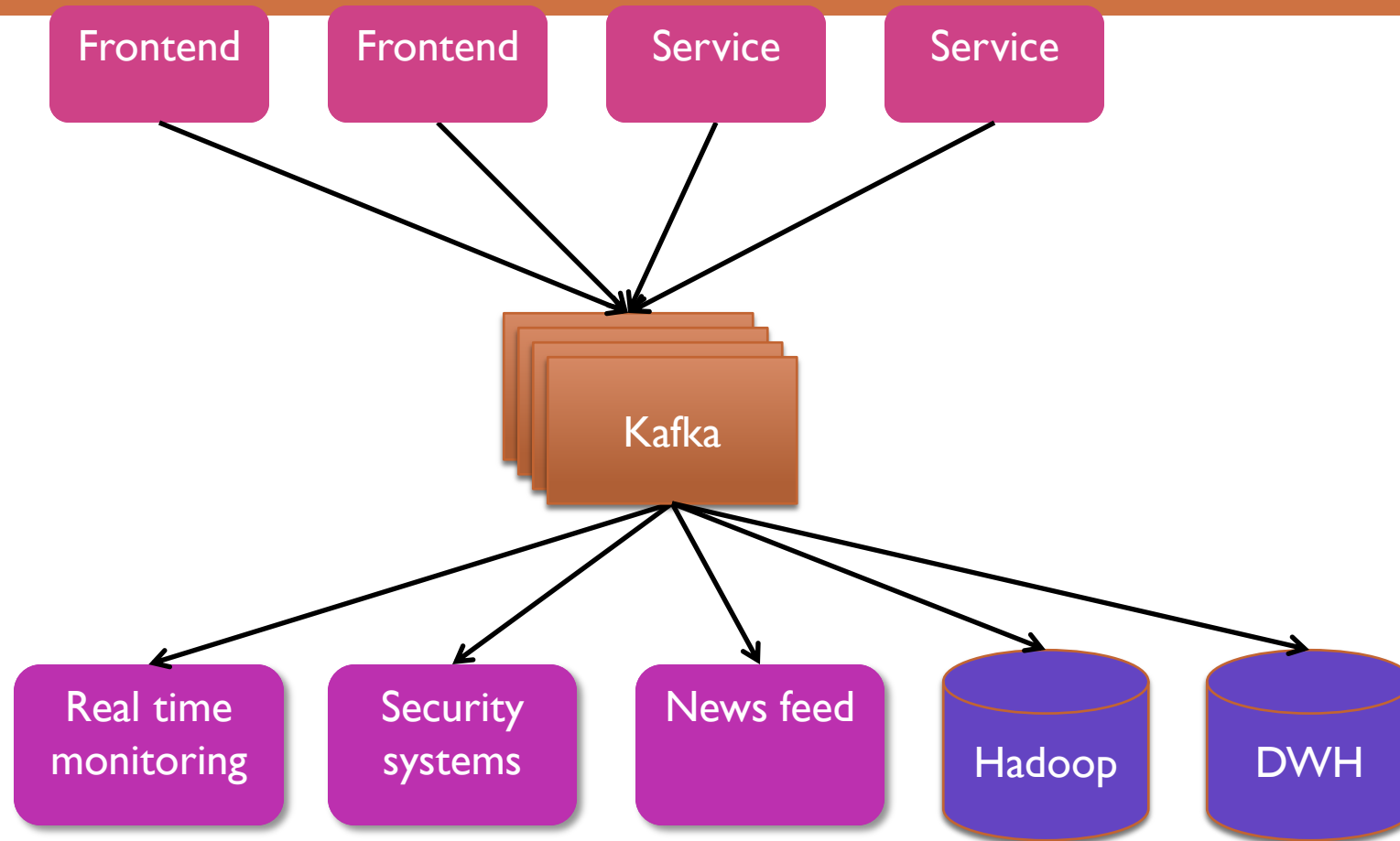
Real-time $\neq$ *nanosecs* or *μ secs*

Real-time means *explicit* or *implicit* time constraints

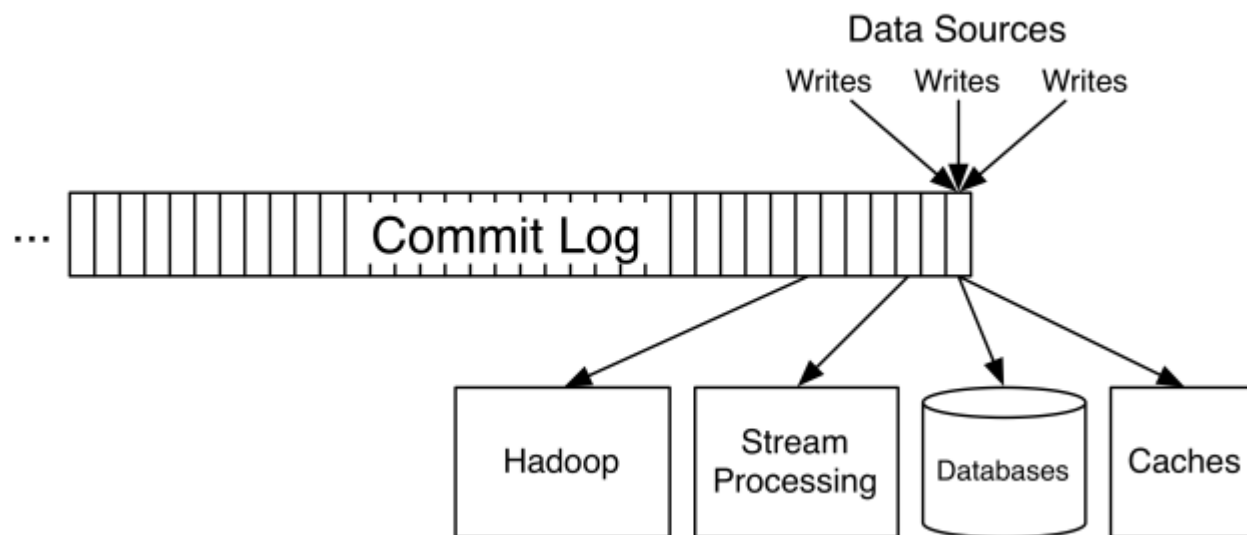
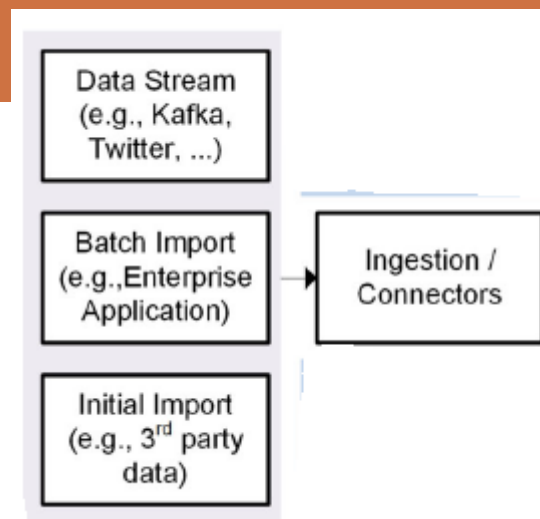
PUB/SUB



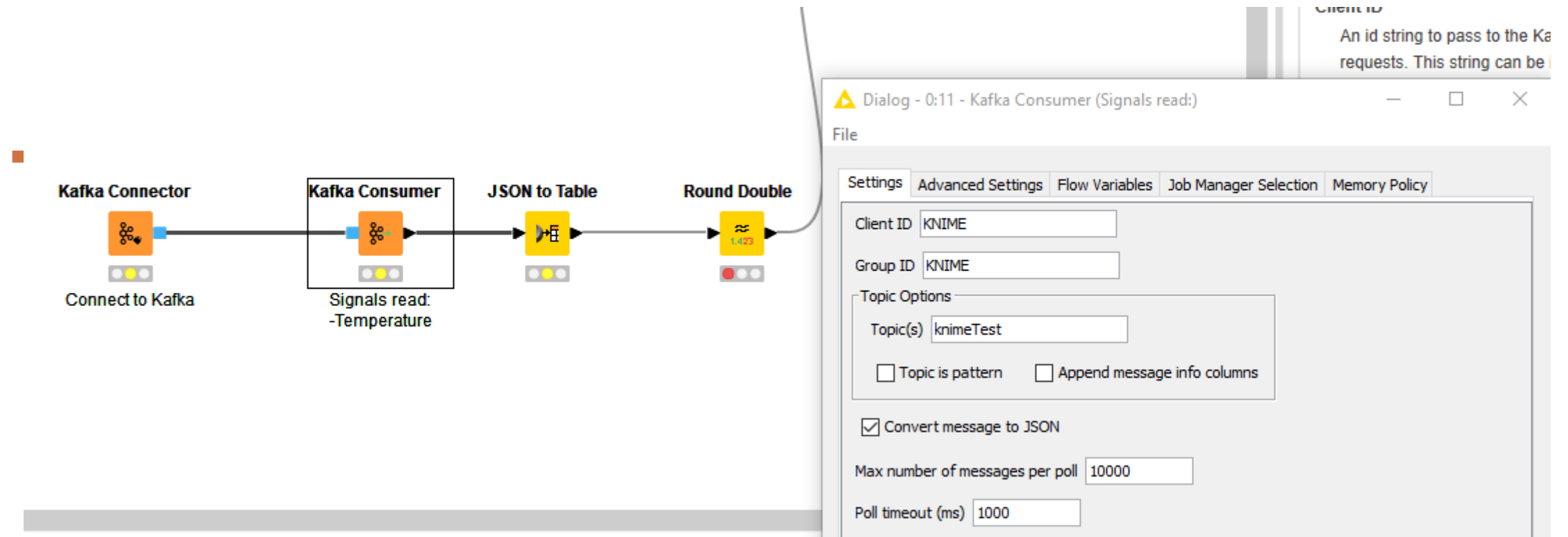
TYPICAL EXAMPLE



KAFKA



KAFKA FOR KNIME



KAFKA FOR TALEND

tKafkaConnection_1

Basic settings

Advanced settings

Dynamic settings

View

Documentation

Version *

Connection

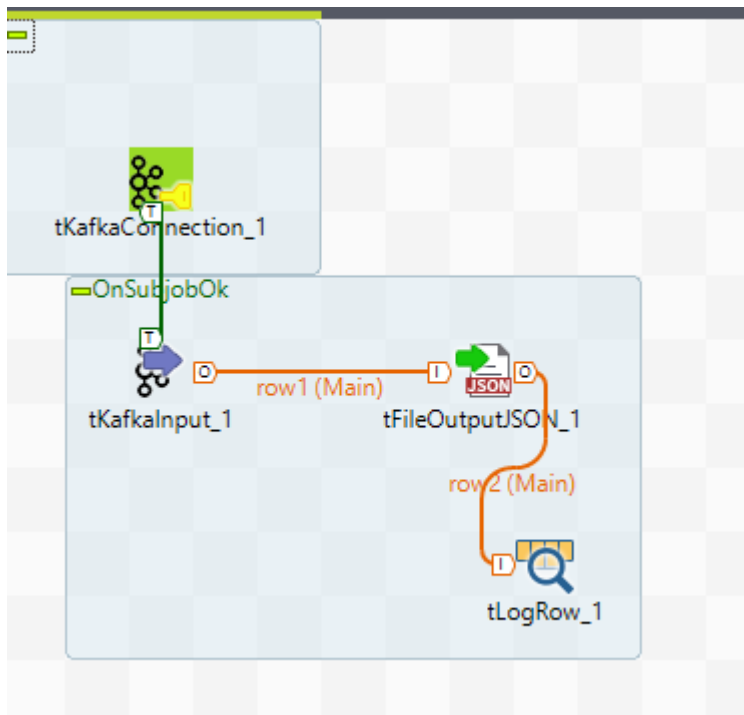
Zookeeper quorum list

Broker list

Security

☐ Use SSL/TLS

☐ Use kerberos authentication



tKafkaInput_1

Basic settings

Advanced settings

Dynamic settings

View

Documentation

Output type Schema Edit schema ...

☒ Use an existing connection *

Topic name

Consumer group id

Offset management

☐ Reset offsets on consumer group

New consumer group starts from

☒ Auto-commit offsets Interval (ms)

Message batches

☐ Stop after a maximum total duration (ms)

☐ Stop after receiving a maximum number of messages

☒ Stop after maximum time waiting between messages (ms)

KAFKA FOR PYTHON

- install Kafka-Python
- Initialize the producer

```
producer = KafkaProducer(bootstrap_servers=['localhost:9092'],
                           value_serializer=lambda x:
                               dumps(x).encode('utf-8'))
```
- Producer sends to topic queue

```
producer.send('topic', value=data)
```
- Initialize the consumer (other script)

```
consumer = KafkaConsumer( topic', bootstrap_servers=['localhost:9092'],
                           auto_offset_reset='earliest', enable_auto_commit=True, group_id='my-group',
                           value_deserializer=lambda x: loads(x.decode('utf-8'))))
```
- The consumer

```
for message in consumer:
    message = message.value
    //do something
```



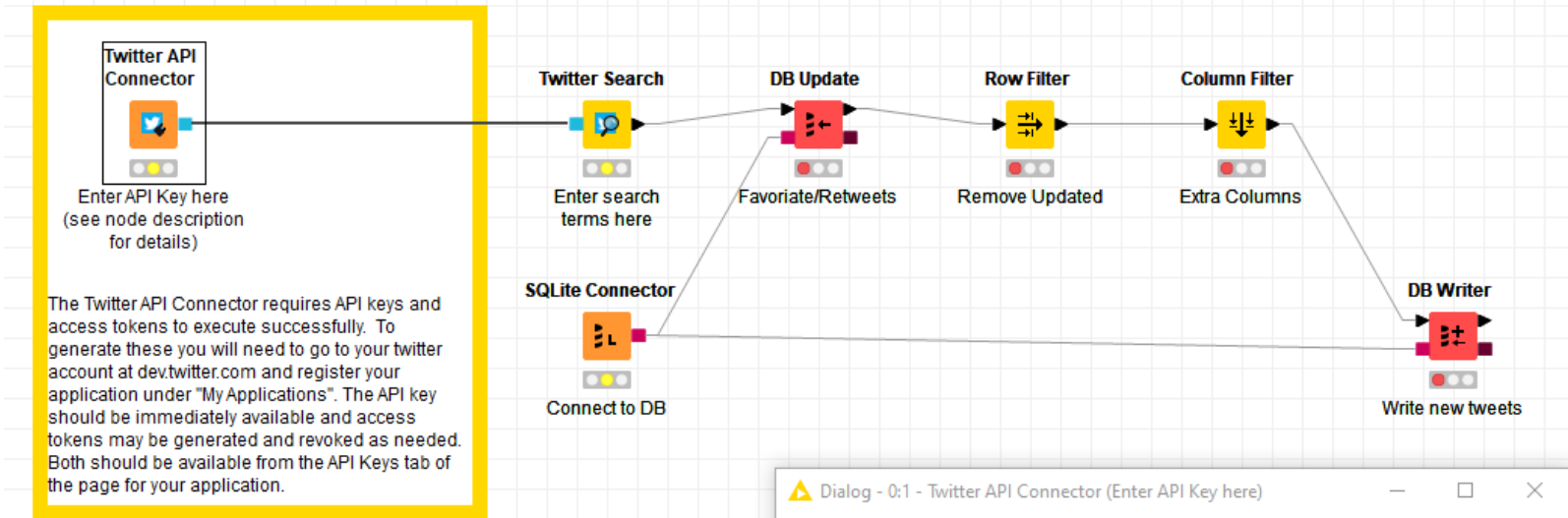
TWITTER API



TWITTER API

- The Twitter API can be used to programmatically retrieve and analyze data, as well as engage with the conversation on Twitter.
- This API provides access to a variety of different resources including the following:
 - Tweets
 - Users
 - Direct Messages
 - Lists
 - Trends
 - Media
 - Places
- Academic Research account
 - This track provides qualified academic researchers access to elevated access and enhanced functionality, including access to the full-archive search endpoint, a higher monthly Tweet cap, and enhanced filtering capabilities with the filtered stream and recent search endpoints.
 - With these credentials you can obtain the API Key, secret key... to start to use it

TWITTER AND KNIME



Dialog - 0:2 - Twitter Search (Enter search)

File

Config | Flow Variables | Job Manager Selection | Memory Policy

query: KNIME

search for: recent

number of rows: 5,000

number of retries: 0

☒ return intermediate results

fields

tweets	Tweet Tweet ID Time Favorited Count Retweeted Count	users	User (Screen Name) Profile Image Name ID Description
--------	--	-------	---

OK Apply Cancel ?

Dialog - 0:1 - Twitter API Connector (Enter API Key here)

File

Credentials | Flow Variables | Job Manager Selection

API key: <your-api-key>

API secret:

Access token: <your-access-token>

Access token secret:

OK Apply Cancel ?

TWITTER API

- Some relevant endpoint
 - Recent tweet
 - 100 by request in the last week
 - fullHistory
 - 500 by request since 2006
 - Streaming
 - 1000 rules for finding interesting tweets



SCRAPER



THE PROBLEM

- How to find new researchers in a given field of computer science ready to become associate professor
 - Not early stage researchers
 - National scientific qualification
- The key entry point
 - https://abilitazione.miur.it/public/pubblicarisultati_2018.php

SCRAPER

- Scraping is the process to extract data from a system without using API
 - It can be used also for legacy interface (bank systems)
- Web scraping is the process to extract data from web pages
- Is it legal?
 - Grey area
- Two examples
 - <https://asn18.cineca.it/pubblico/miur/esito-abilitato/01%252FB1/1/6>
 - <https://groups.google.com/g/spaghettiiopendata>

WEB SCRAPING WITH TOOLS

- There are different types of tools for web scraping
 - Browser extension
 - Open web scraper
 - Web scraper tool
 - Mozenda
 - Zyte

OPEN WEB SCRAPER

Settore Concorsuale 01/B1 - I Fascia - Sesto Quadrimestre

Cognome	Nome	Abilitato	Validità Abilitazione
ANSELMO	Marcella	Si	Dal 29/04/2021 al 29/04/2030
BERTINI	Marco	Si	Dal 29/04/2021 al 29/04/2030

P

C

S

Done selecting

Elementi

Console

Origini

Rete

Prestazioni

Memoria

Applicazione

Sicurezza

Lighthouse

Web Scraper

Sitemaps

Sitemap abilita

Create new sitemap

Id

test

Type

Table

Selector

Select

Element preview

Data preview

Header row selector

Select

Element preview

Data rows selector

Select

Element preview

Multiple

☒

Table columns

Column

Result key

Include into result

Table columns

Column	Result key	Include into result
Cognome	Cognome	☒
Nome	Nome	☒
Abilitato	Abilitato	☒
Validità Abilitazione	Validità Abilitazione	☒

Data Preview

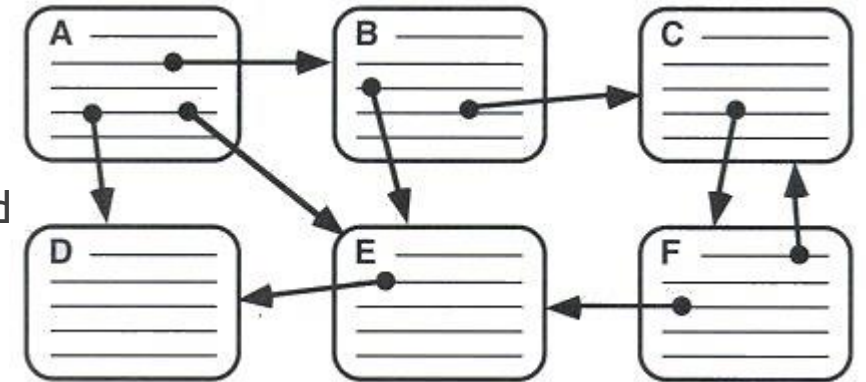
Cognome	Nome	Abilitato	Validità Abilitazione
ANSELMO	Marcella	Si	Dal 29/04/2021 al 29/04/2030
BERTINI	Marco	Si	Dal 29/04/2021 al 29/04/2030
BIGGIO	Battista	Si	Dal 29/04/2021 al 29/04/2030
BODEI	Chiara	Si	Dal 29/04/2021 al 29/04/2030

WEB SCRAPER

- Sometime existing tools are not able to complete the scraping processing
 - Licence limitation
 - Complexity of web pages
 - Other (see later)
- We need to know html

HTML

- A web page is composed by different objects or resources
- A resource is a file identified by an URL.
- HTML is the markup language used to create hypertext
 - Create link, insert image, table and so on
- A markup language is a special type of language where data are described
- An html page can be view in a browser



HTML EXAMPLE

```
<!doctype html>
<html>
  <head>
    <title>Basic HTML page</title>
  </head>
  <body>
    <!-- only the content of this tag will be shown-->
    Hello world
  </body>
</html>
```

HTML

- `<TAG> CONTENT </TAG>`
open close
- `<u>this in an example</u>` will be rendered as
this in an example
- A tag may include one or more attributes
- `<tag att1="val1" att2="val2">....</tag>`

HTML

```
<table> <caption>    <p>Table caption</p> </caption>
  <thead> ← Table head
    <tr> ← Table Row
      <th>Column 1</th>
      <th>Column 2</th></tr>  </thead>
  <tfoot> ← Table foot
    <tr> <td>Total 1</td> ← Table data
      <td>Total 2</td> </tr>  </tfoot>
  <tbody> ← table body
    <tr> <td>Data 1,1</td> <td>Data 1,2</td> </tr>
    <tr> <td>Data 2,1</td> <td>Data 2,2</td> </tr>
    ...
  </tbody>
</table>
```

Column 1	Column 2
Total 1	Total 2

WEB SCRAPING IN KNIME

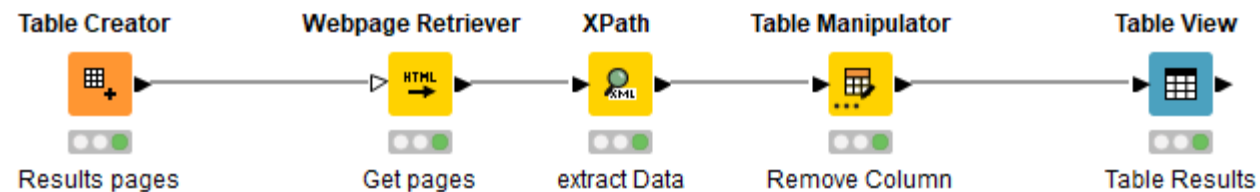
Webpage Retriever



- This node can be used to retrieve webpages by issuing HTTP GET requests and parsing the requested HTML webpages
- By default, the output table will contain a column with the parsed HTML converted into XHTML
- Then you have to extract data you need by means of regular expression, XPATH or similar

WEB SCRAPING WITH KNIME

Extract data from web pages



- Example XPATH query
 - `//table//tr/td[1]` ← Surname
 - `//table//tr/td[2]` ← Name

Show entries Search:

☐	RowID	Cognome	Nome
☐	Row0	ARCAINI	Paolo
☐	Row1	BAIOLETTI	Marco
☐	Row2	BALLIS	Demis
☐	Row3	BASILE	Valerio
☐	Row4	BONETTA	Daniele
☐	Row5	BONNICI	Vincenzo
☐	Row6	BOSCO	Cristina
☐	Row7	BRACCIALI	Andrea
☐	Row8	BRIOLA	Daniela
☐	Row9	BRUNETTI	Sara

Showing 1 to 10 of 247 entries

Previous **1** 2 3 4 5 ... 25 Next

CHAT?

- Could you write the python code for scraping the table described here
<https://asn21.cineca.it/pubblico/miur/esito-abilitato/01%252FB1/2/5>
- Try....it is work!



Certainly, you can scrape a table from a web page using Python with the help of libraries like `requests` and `BeautifulSoup`. To scrape the table from the URL you provided, you can use the following code:

```
python Copy code

import requests
from bs4 import BeautifulSoup

# Define the URL of the webpage
url = "https://asn21.cineca.it/pubblico/miur/esito-abilitato/01%252FB1/2/5"

# Send a GET request to the URL
response = requests.get(url)

# Check if the request was successful
if response.status_code == 200:
    # Parse the HTML content of the page
    soup = BeautifulSoup(response.text, 'html.parser')

    # Find the table you want to scrape by inspecting the HTML source
    table = soup.find('table')

    # Iterate through the rows of the table and extract the data
    for row in table.find_all('tr'):
        columns = row.find_all('td')
        if columns:
            # Extract and print the data in each column
            data = [column.get_text(strip=True) for column in columns]
            print("\t".join(data))
else:
    print("Failed to retrieve the webpage. Status code:", response.status_code)
```

WEB SCRAPER IN PYTHON

```
In [8]: from bs4 import BeautifulSoup
```

```
In [12]: url = "https://asn18.cineca.it/pubblico/miur/esito-abilitato/01%252FB1/1/6"
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')
print(soup.prettify())
```

```
In [24]: myTable=soup.find_all("tr")
for tr in myTable:
    for child in tr.children:
        for td in child:
            print(td)
```

Cognome

Nome

Abilitato

Validità Abilitazione

ANSELMO

Marcella

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8"/>
    <meta content="width=device-width, initial-scale=1" name="viewport"/>
    <title>
      ASN - Risultati ASN 01/B1 - INFORMATICA
    </title>
    <script src="https://asn18.cineca.it/js/69c6871.js" type="text/javascript">
    </script>
    <link href="https://fonts.googleapis.com/css?family=Roboto+Condensed" rel="stylesheet" type="text/css"/>
    <link href="https://asn18.cineca.it/css/a777d22.css" rel="stylesheet">
    <!-- HTML5 Shim and Respond.js IE8 support of HTML5 elements and media queries -->
    <!-- WARNING: Respond.js doesn't work if you view the page via file:// -->
    <!--[if lt IE 9]>
      <script src="https://oss.maxcdn.com/libs/html5shiv/3.7.0/html5shiv.js"></script>
      <script src="https://oss.maxcdn.com/libs/respond.js/1.4.2/respond.min.js"></script>
    </if-->
```

It is enough for my problem?

OUR NEEDS

- We need to find if qualified researchers are already hired in a italian university
- <https://cercauniversita.cineca.it/php5/docenti/cerca.php>

- Not a read only page
- We must to interact

Ruolo:
Tutti i ruoli

☒ Tutti ☐ Confermati ☐ Non confermati

Cognome: Nome:

☐ Cognome esatto ☒ Parte del cognome ☐ Inizio del cognome

Genere: ☒ Tutti ☐ Maschile ☐ Femminile

Ateneo:
Tutti gli atenei

Servizio in altro ateneo al 100% (convenzioni Art.6, comma 11, Legge 240/10): ☐ 100%

Facoltà:
Tutte le Facoltà

S.S.D.:
Tutti i settori

Area:
Tutte le aree

Macrosettore:
Tutti i macrosettori

Settore concorsuale:
Il menù a tendina presenta i settori concorsuali determinati dal Decreto Ministeriale n. 855 30 ottobre 2015, GU n. 271 del 20-11-2015 - Suppl. Ordinario n.63. Per ricercare docenti inquadrati in settori concorsuali soppressi è necessario selezionare la voce 'Tutti'
Tutti i settori concorsuali

Situazione:
ad oggi

HTML FORM

An HTML form is used to collect user input.

```
<form action="URI" method="GET|POST">
```

```
  <label for="name">Name</label>
```

```
  <input id="name" type="text">
```

```
  <input type="submit" value="OK">
```

```
</form>
```

Name

HTML FORM

Type	Description
<code><input type="text"></code>	Displays a single-line text input field
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices)
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices)
<code><input type="submit"></code>	Displays a submit button (for submitting the form)
<code><input type="button"></code>	Displays a clickable button

HTML FORM

<p>Choose your favorite Web language:</p>

<form>

<input type="radio" id="html" name="fav_language" value="HTML">

<label for="html">HTML</label>

<input type="radio" id="css" name="fav_language" value="CSS">

<label for="css">CSS</label>

<input type="radio" id="javascript" name="fav_language" value="JavaScript">

<label for="javascript">JavaScript</label>

</form>

- ☐ HTML
- ☒ CSS
- ☐ JavaScript

HTML FORM

<form>

<input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">

<label for="vehicle1"> I have a bike</label>

<input type="checkbox" id="vehicle2" name="vehicle2" value="Car">

<label for="vehicle2"> I have a car</label>

<input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">

<label for="vehicle3"> I have a boat</label>

</form>

- ☐ I have a bike
- ☐ I have a car
- ☐ I have a boat

WEBSCRAPER IN PYTHON: POST

- There are two methods for posting HTML Form: get and post

- GET

- All parameters are sent at the end of URI

- `https://URI?VarName1=VarValue1&VarName2=VarValue2....`

- In this case you have to compose the request with appropriate parameters

- ```
requests.get("https://api.spotify.com/v1/search?query=90s&offset=20&limit=20&type=playlist")
```

- POST

- You have to send parameters in a different ways

- ```
requests.post("http://whatever.com/url/to/something", { "param1": "val1", "param2": "val2" })
```

CHAT

- Now I want to scrape the following page <https://cercauniversita.cineca.it/php5/docenti/cerca.php> the page has some HTML form and I need to create a function with two parameters, name and surname and a payload json data including attribute "conferma" equal to 2...



To scrape the page you provided, which contains an HTML form, and create a function that takes name and surname as parameters and sends a POST request with a JSON payload, you can use the `requests` library. Here's a Python function that accomplishes this:

```
python Copy code

import requests

def scrape_university_data(name, surname):
    # URL of the target page
    url = "https://cercauniversita.cineca.it/php5/docenti/cerca.php"

    # Define the payload as a JSON object
    payload = {
        "name": name,
        "surname": surname,
        "conferma": 2
    }

    # Send a POST request with the JSON payload
    response = requests.post(url, json=payload)

    # Check if the request was successful
    if response.status_code == 200:
        # Parse and print the response content
        print(response.text)
    else:
        print("Failed to retrieve the webpage. Status code:", response.status_code)

# Example usage of the function
name = "John"
surname = "Doe"
scrape_university_data(name, surname)
```

OUR PROBLEM

Ruolo:
Tutti i ruoli

☒ Tutti ☐ Confermati ☐ Non confermati

Cognome: Nome:

☐ Cognome esatto ☒ Parte del cognome ☐ Inizio del cognome

Genere: ☒ Tutti ☐ Maschile ☐ Femminile

Ateneo:
Tutti gli atenei

Servizio in altro ateneo al 100% (convenzioni Art.6, comma 11, Legge 240/10): ☐ 100%

Facoltà:
Tutte le Facoltà

S.S.D.:
Tutti i settori

Area:
Tutte le aree

Macrosettore:
Tutti i macrosettori

Settore concorsuale:
Il menù a tendina presenta i settori concorsuali determinati dal Decreto Ministeriale n. 855 30 ottobre 2015, GU n. 271 del 20-11-2015 - Suppl. Ordinario n.63. Per ricercare docenti inquadrati in settori concorsuali soppressi è necessario selezionare la voce 'Tutti'

Tutti i settori concorsuali

Situazione:
ad oggi

It is enough?

```
def get_prof_name(cognome, nome):  
    token = {'qualifica': '**',  
            'conferma': '2',  
            'cognome': cognome,  
            'nome': nome,  
            'radiogroup': 'I',  
            'genere': 'A',  
            'universita': '00',  
            'facolta': '00',  
            'settore': '0000',  
            'area': '0000',  
            'macro': '0000',  
            'settorec': '0000',  
            'situazione_al': '0',  
            'vai': 'Invio'  
            }  
  
    return token
```

```
for rowA in abili.iterrows():  
    payload= get_prof_name(rowA.Cognome,rowA.Nome)  
    response_post = requests.post(url, data=payload)  
    soup = BeautifulSoup(response_post.text, 'html.parser')
```

OUR PROBLEM

- I need a qualified researchers in the field of data management.
- How can I understand the main research topic of a given researchers?
- Publications
- How in an automatic way, I can extract the topic from publications
 - Title
 - Venues

2021	
[c68]	Stefano Fiorini, Michele Ciavotta, Andrea Maurino: A multi-criteria algorithm for automatic detection of city commun 1271
[i2]	Stefano Fiorini, Michele Ciavotta, Andrea Maurino: Listening to the city, attentively: A Spatio-Temporal Attention Boo Short-Term Flow Prediction Problem. CoRR abs/2103.00983 (2021)
2020	
[c67]	Stefano Fiorini, Giorgio Pilotti, Michele Ciavotta, Andrea Maurino: 3D-CLoST: A CNN-LSTM Approach for Mobility Dynamics Prediction BigData 2020: 3180-3189
[c66]	Marco Cremaschi, Alessandra Siano, Roberto Avogadro, Ernesto Jiménez STILTool: A Semantic Table Interpretation evalUation Tool. ESWC (S
[c65]	Marco Valzelli, Andrea Maurino, Matteo Palmonari: A Fine-grained Access Control Model for Knowledge Graphs. ICETE (
[c64]	Giuseppe De Pasquale, Blerina Spahiu, Pietro Ducange, Andrea Maur Towards Automatic Classification of Sheet Music. SEBD 2020: 266-2
[-] 2010 – 2019	

2019	
[j21]	Vincenzo Cutrona, Federico Bianchi, Michele Ciavotta, Andrea M On the composition and recommendation of multi-feature paths: i approach. Geoinformatica 23(3): 353-373 (2019)
[j20]	Blerina Spahiu, Andrea Maurino, Robert Meusel: Topic profiling benchmarks in the linked open data cloud: Issues a Semantic Web 10(2): 329-348 (2019)
[j19]	Anisa Rula, Matteo Palmonari, Simone Rubinacci, Axel-Cyrille Ngon , Andrea Maurino, Diego Esteves: TISCO: Temporal scoping of facts. J. Web Semant. 54: 72-86 (2019)

[-] Person information

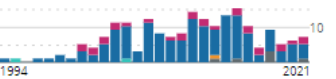
- affiliation: University of Washington, Seattle, USA

[+] Other persons with a similar name

2021	
[j29]	Arianna Blasi, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, Antonio Carzaniga: MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation. J. Syst. Softw. 181: 111041 (2021)
[j28]	Daming Zou, Jingjing Liang, Yingfei Xiong, Michael D. Ernst, Lu Zhang: An Empirical Study of Fault Localization Families and Their Combinations. IEEE Trans. Software Eng. 47(2): 332-347 (2021)
[c150]	Rashmi Mudduluru, Jason Waataja, Suzanne Millstein, Michael D. Ernst: Verifying Determinism in Sequential Programs. ICSE 2021: 37-49
[c149]	Rashmi Mudduluru, Jason Waataja, Suzanne Millstein, Michael D. Ernst: Verifying Determinism in Sequential Programs. ICSE (Companion Volume) 2021: 213-214
[c148]	Zhen Zhang, Yu Feng, Michael D. Ernst, Sebastian Porst, Isil Dillig: Checking conformance of applications against GUI policies. ESEC/SIGSOFT FSE 2021: 95-106
[c147]	Martin Kellogg, Narges Shadab, Manu Sridharan, Michael D. Ernst: Lightweight and modular resource leak verification. ESEC/SIGSOFT FSE 2021: 181-192
[i6]	Sahil Verma, Michael D. Ernst, René Just: Removing biased data to improve fairness and accuracy. CoRR abs/2102.03054 (2021)
2020	
[j27]	Ivan Beschastnikh, Perry Liu, Albert Xing, Patty Wang, Yuriy Brun, Michael D. Ernst: Visualizing Distributed System Executions. ACM Trans. Softw. Eng. Methodol. 29(2): 9:1-9:38 (2020)
[c146]	Annie Louis, Santanu Kumar Dash, Earl T. Barr, Michael D. Ernst, Charles Sutton: Where should I comment my code?: a dataset and model for predicting locations that need comments. ICSE (NIER) 2020: 21-24
[c145]	Martin Kellogg, Manli Ran, Manu Sridharan, Martin Schäf, Michael D. Ernst: Verifying object construction. ICSE 2020: 1447-1458

[-] Refine list

showing all 188 records



refine by search term

refine by type

- ☒ Journal Articles (only)
 - ☒ Conference and Workshop Papers (only)
 - ☒ Parts in Books or Collections (only)
 - ☒ Editorship (only)
 - ☒ Informal Publications (only)
- select all | deselect all

refine by coauthor

Yuriy Brun (21)
David Notkin (15)
Adam Kiezun (13)
René Just (12)
Ivan Beschastnikh (11)
Shay Artzi (11)
Zachary Tatlock (10)
Werner Dietl (9)
Sai Zhang (9)
Jeff H. Perkins (8)
220 more options

refine by venue

ICSE (32)
ISSTA (23)
ECOOP (9)
ASE (9)
SIGSOFT FSE (8)
OOPSLA (8)
OOPSLA Companion (8)

WEB SCRAPER

- In some case web scraping by reading the page source is it not enough
- <https://dblp.org/pid/41/3166.html>
- Selenium
 - It is a library for controlling web browsers through programs and performing browser automation
 - You can, via python, lunch an instance of your browser, get an interested page, fill in forms and select link or button

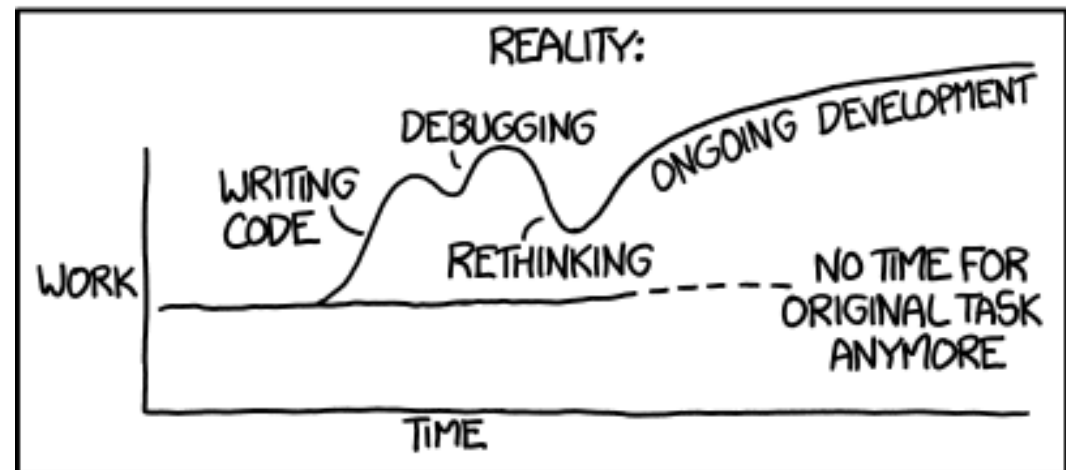
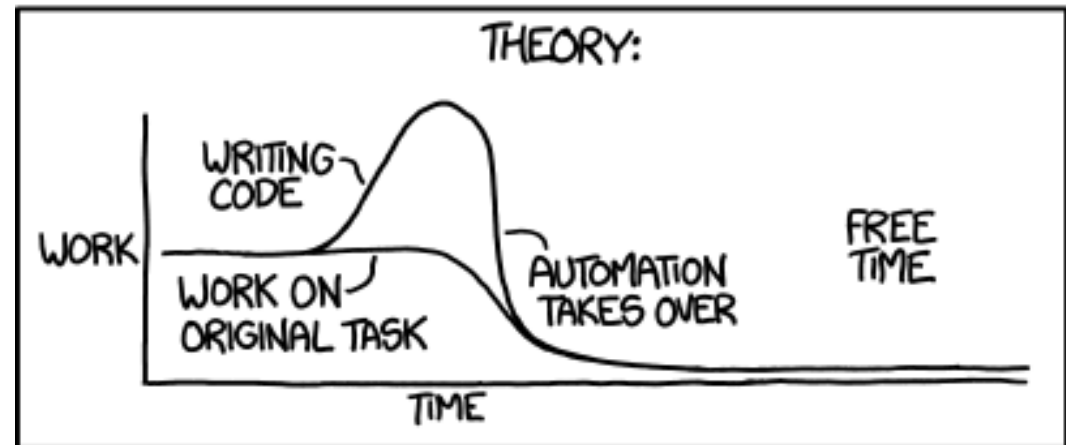
SELENIUM

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import Select
from selenium.webdriver.support.ui import WebDriverWait
...
driver = webdriver.Chrome()
driver.get(«URL»)
    driver.find_element_by_name(«Fool»)
```

BE AWARE ABOUT IT

- Pay attention to the time it takes to create an automated solution for your needs

"I SPEND A LOT OF TIME ON THIS TASK.
I SHOULD WRITE A PROGRAM AUTOMATING IT!"



CONCLUSION

- Data acquisition is the second step in a DS pipeline
- According where data are and their licence you can
 - Download
 - API
 - Scraper
- In some case you are not allowed to use one of these methods
- You can use
 - Dedicated tools
 - General purpose data platform
 - Python programs



EXERCISE



EXAMPLES

- For the following examples
 - Complete the request
 - Find right data sources (or web site)
 - Find the most appropriate data acquisition strategy
- Music
 - How it is changed Italian music in the last 5 years in terms of lyrics, sound and so on
- Social listening.
 - Is it possible to understand the evolution of a football match by means of the volume of social interaction?
- Smart Cities
 - How to predict the use of bike sharing services free floating or fixed station



A BIKE SHARING EXAMPLE



A REAL EXAMPLE (MOBIKE FREE FLOATING BIKE SHARING IN MILAN)

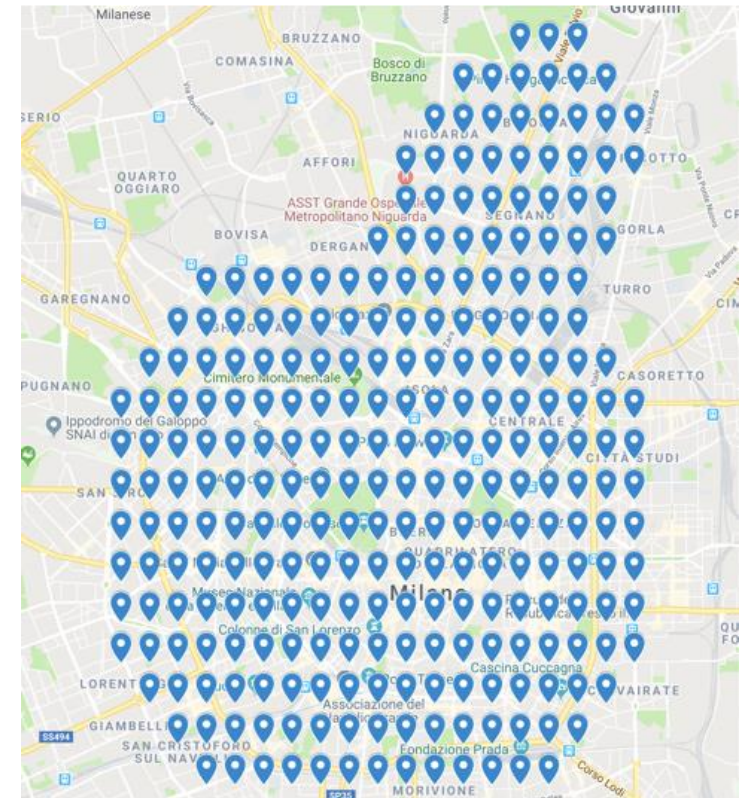
A mobile app allows users to pick up a bike

The app allow users to ask if there is an available bike in a range of 500 meters by using the gps position of the app

By simulating multiple requests it is possible to discover how many bikes are available in all the city

```
{
  "code": 0,
  "message": "",
  "bike": [
    {
      "distId": "A050004985",
      "distX": 9.188465,
      "distY": 45.464669,
      "distNum": 1,
      "distance": "53",
      "bikelds": "A050004985#",
      "biketype": 1,
      "type": 0,
      "boundary": null,
      "operateType": 981
    },
    ...
  ],
  "mpl": [],
  "biketype": 0,
  "radius": 500,
  "autoZoom": false,
  "hasRedPacket": 0
}
```

One request each 5 minutes for about 260 points

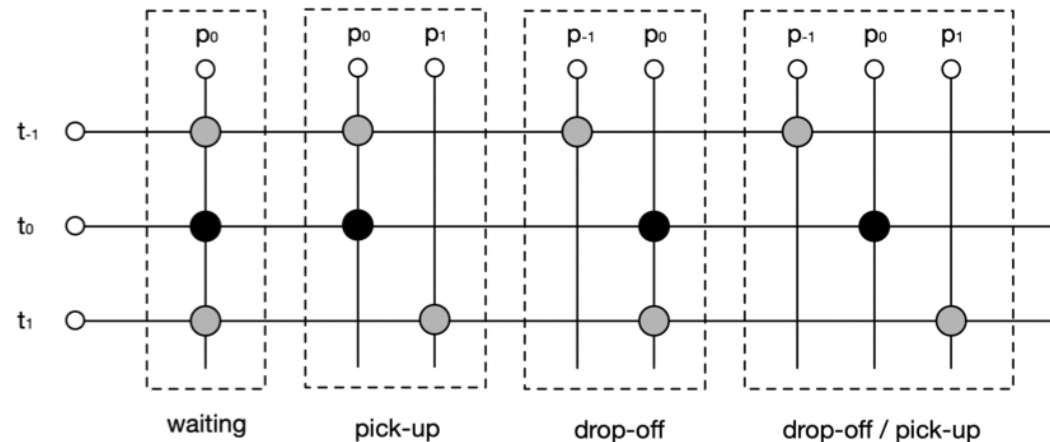


FROM RAW DATA TO INFORMATION

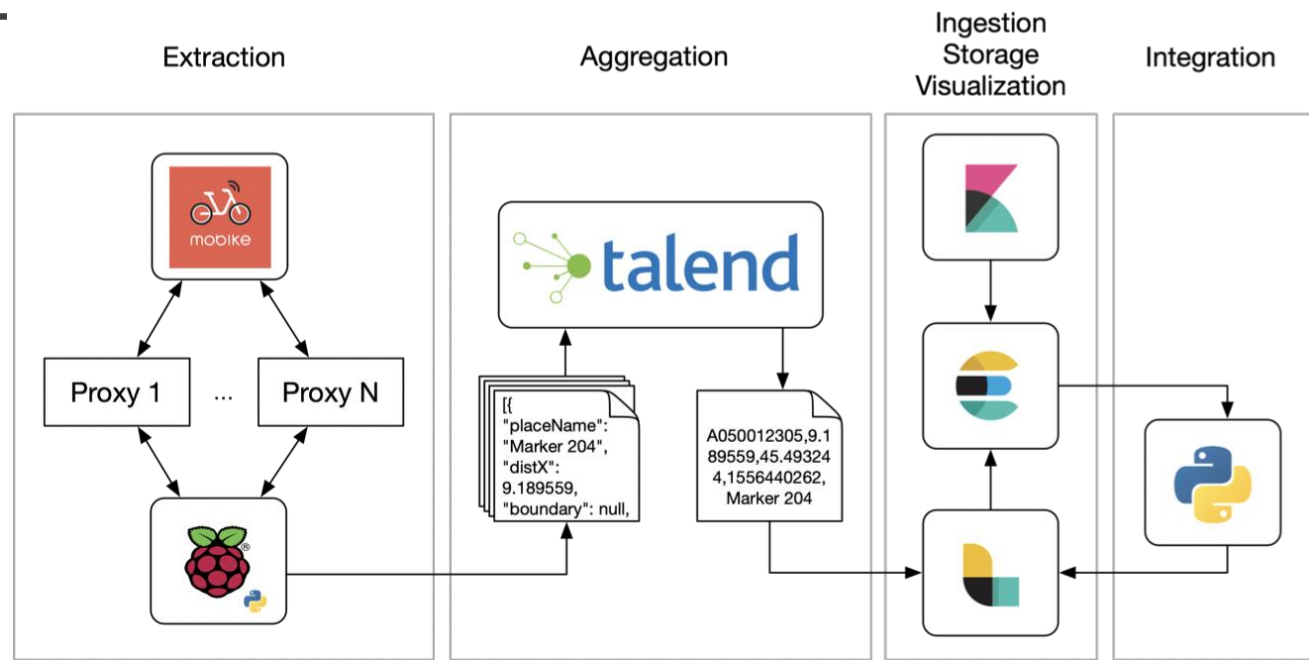
We are interested to discovery the trip of each bike but we have only points.

We classified (ex post) collected points as:

- *waiting* is a bike position is the same with respect to the previous and next point captured for the same bike;
- *pick-up* is the point is the same of the previous point but different from the next one;
- *drop-off* if the point is the same of the next one but different wrt the previous;
- *drop-off / pick-up*.



SOFTWARE ARCHITECTURE



Hardware

Software



Broadcom 1.4GHz
Micro SD 64GB
1GB SDRAM



Intel Core i7 2,7 GHz
SSD 512GB
16GB RAM



3.6.5



7.1.1

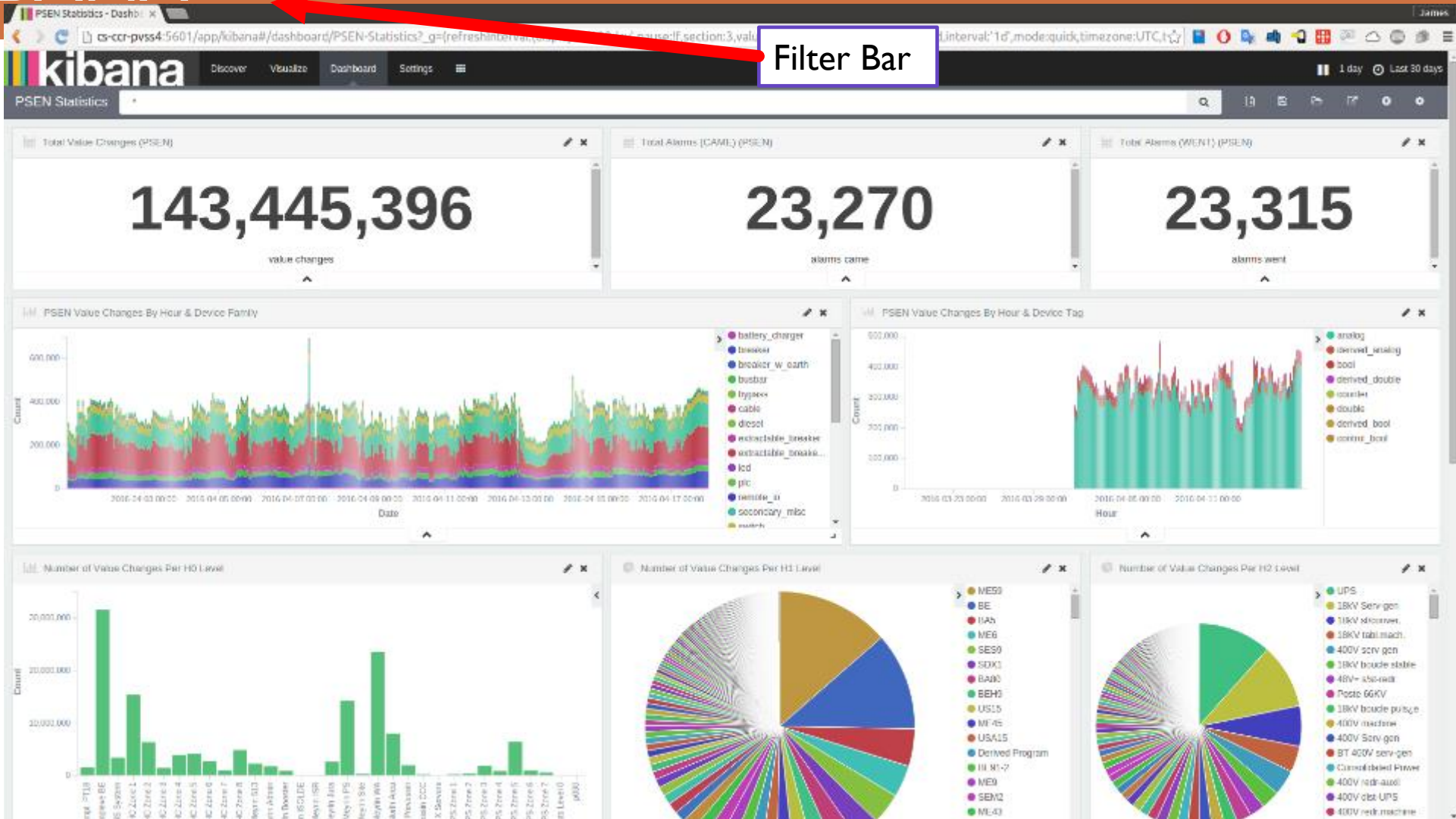


7.0.0

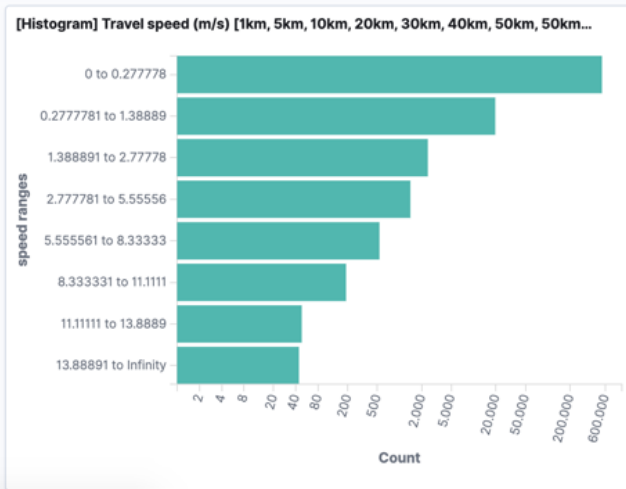
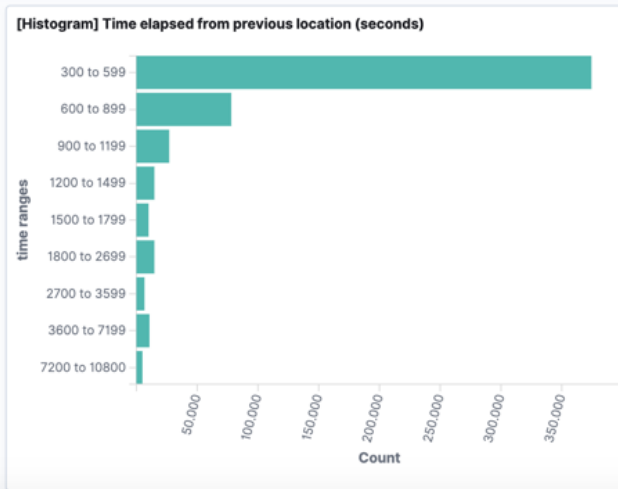
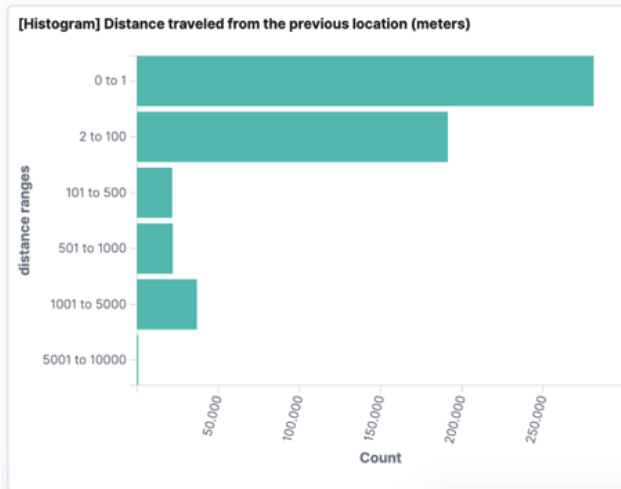
KIBANA

Current Time Period

Filter Bar



DATA QUALTY ERRORS



DATA INGESTION, STORAGE, VISUALIZATION (2 WEEKS)



Lh

8'853'878

Documnts

2,6GB

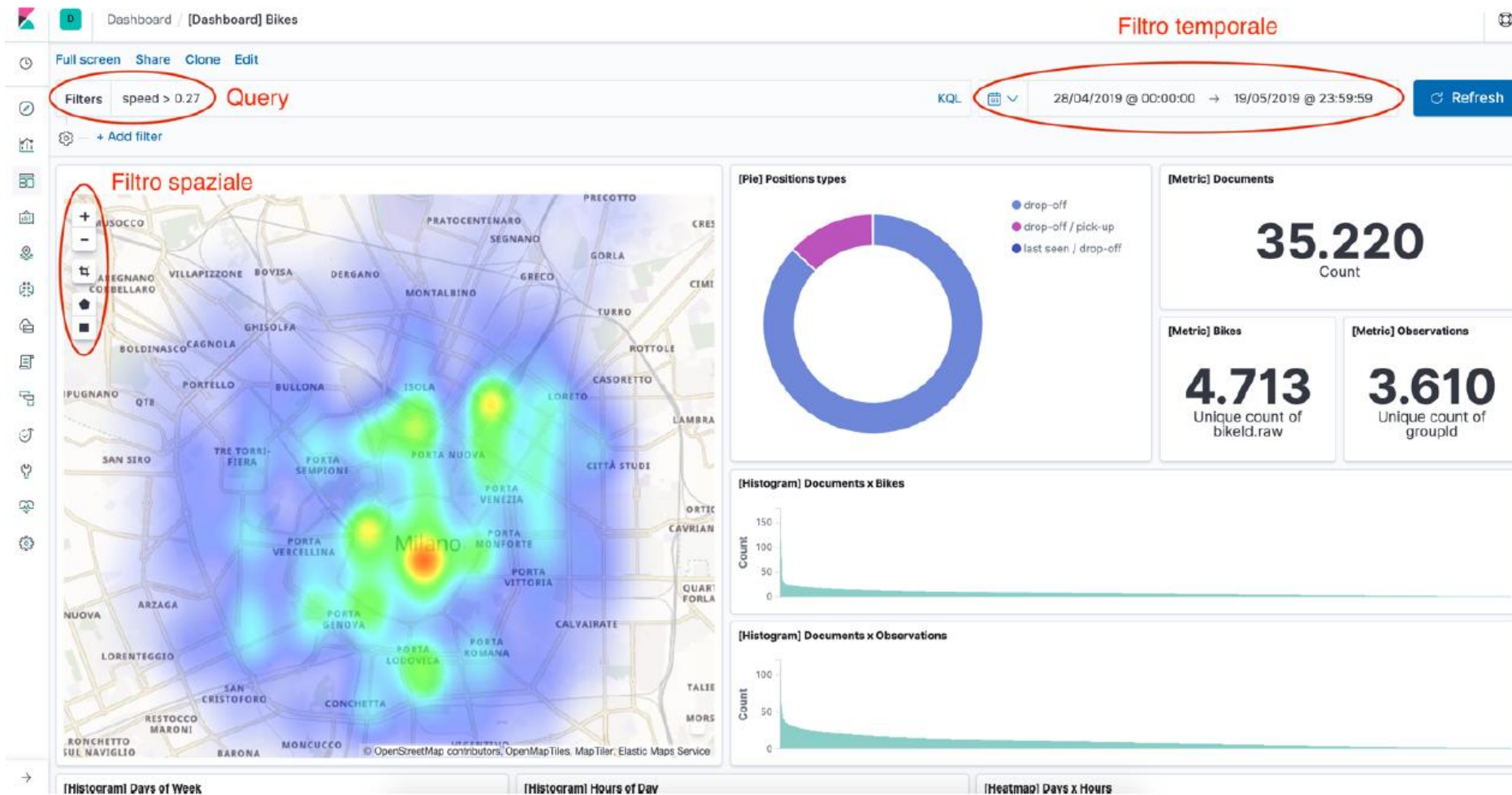
3939

Osservazioni

5'365

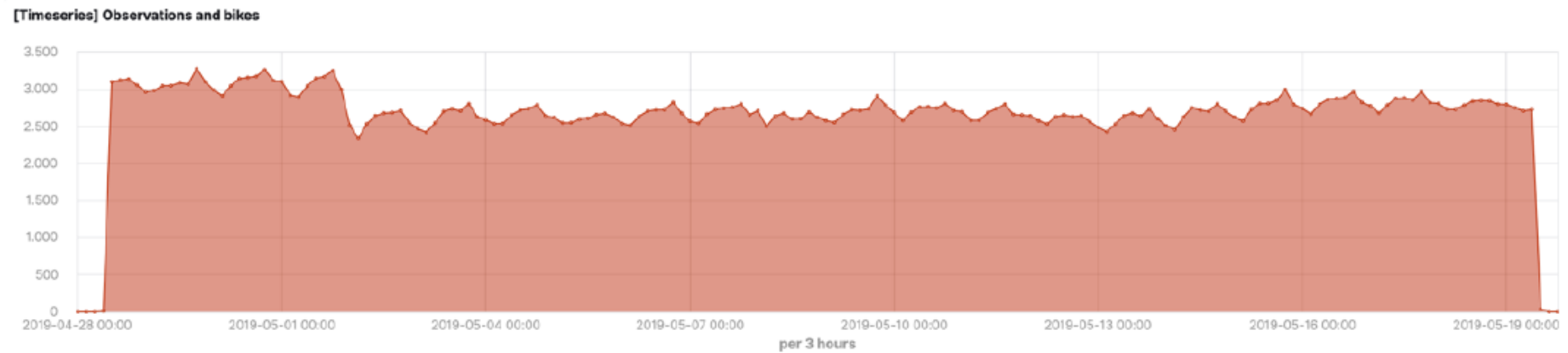
Biciclette univoche

RESULTS



RESULTS

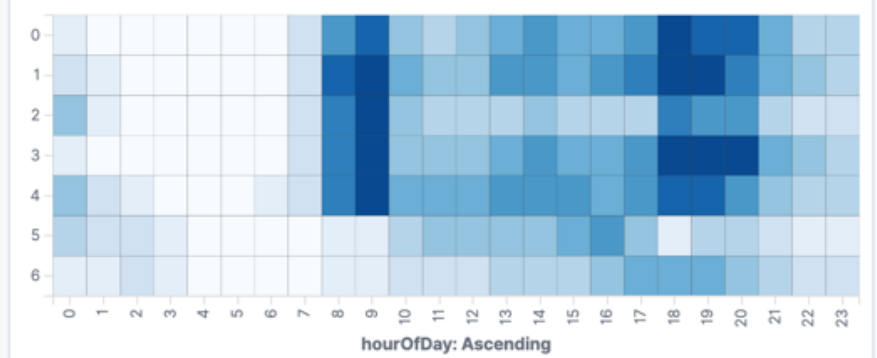
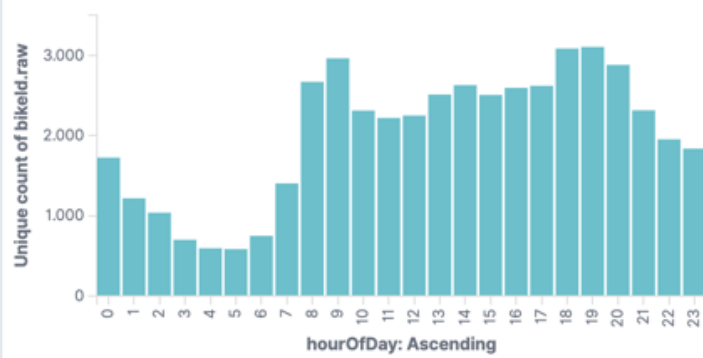
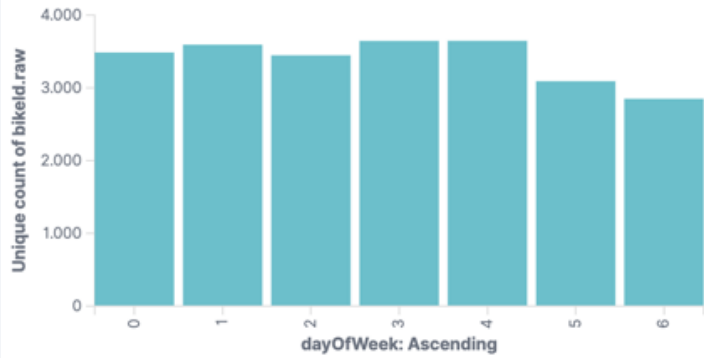
- Salone del mobile and easter Holiday



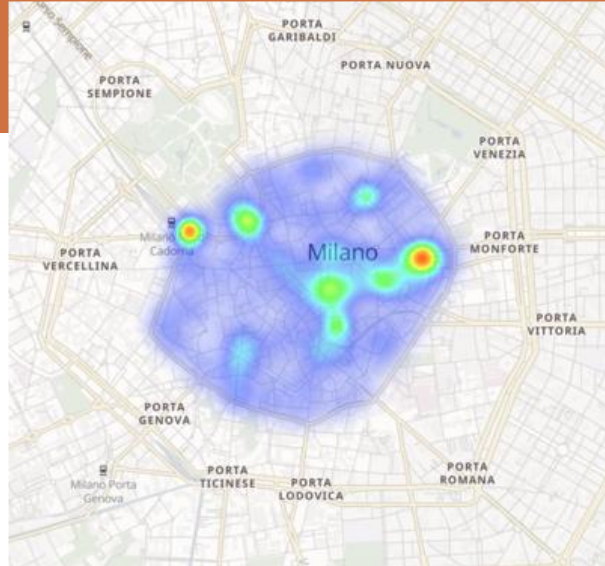
RESULTS



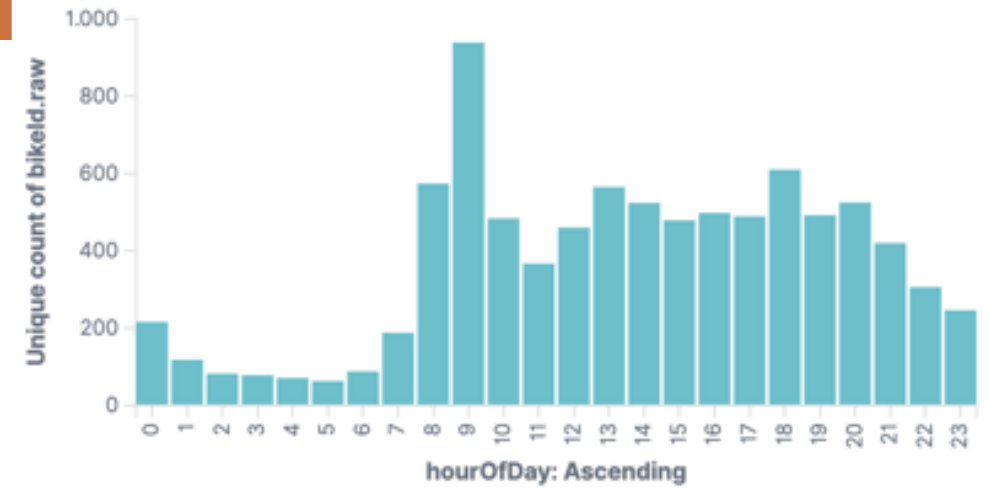
RESULTS



RESULTS



[Histogram] Hours of day



[Histogram] Hours of day

